Projeto Quimera - Guia de Implantação com Contenção de Custos

📋 Sumário Executivo

Este guia apresenta uma estratégia completa de implantação do Projeto Quimera otimizada para **contenção de custos**, mantendo a funcionalidade essencial do sistema. O objetivo é reduzir os custos operacionais mensais de **~$800-1200/mês** (implantação completa) para **~$50-150/mês** (implantação econômica), uma redução de aproximadamente **85-90%**.

Comparação de Custos: Arquitetura Completa vs. Econômica

Componente	Arquitetura Completa	Arquitetura Econômica
Servidor de Aplicação	AWS EC2 t3.large ($60/mês)	Contabo VPS M ($7/mês)
Banco de Dados	AWS RDS PostgreSQL ($50/mês)	PostgreSQL local (incluído)
MongoDB	MongoDB Atlas M10 ($57/mês)	MongoDB local (incluído)
Redis	AWS ElastiCache ($30/mês)	Redis local (incluído)
Armazenamento	AWS S3 ($20/mês)	Armazenamento local (incluído)
Banco Vetorial	Pinecone Starter ($70/mês)	ChromaDB local (gratuito)
LLM (GPT-4)	OpenAI API ($200-400/mês)	Modelos locais + API seletiva ($20-50/m
Proxies	BrightData ($500/mês)	Proxies residenciais baratos ($30/mês)
Monitoramento	ELK + Prometheus ($100/mês)	Logs locais + Grafana Cloud (gratuito)
CDN	CloudFlare Pro ($20/mês)	CloudFlare Free (gratuito)
Email/SMS	SendGrid + Twilio ($30/mês)	SMTP gratuito + Telegram (gratuito)
TOTAL	**$1.137 - 1.337/mês**	**$57 - 87/mês**

---

1. Arquitetura Econômica Proposta

1.1. Visão Geral

A arquitetura econômica consolida todos os serviços em um **único servidor VPS de médio porte**, eliminando a necessidade de serviços gerenciados caros. Utilizamos Docker Compose para orquestração local e priorizamos alternativas open-source.

Plain Text

```
┌──────────────────────────────────────────────────────────────────┐
│                     Contabo VPS M                                 │
│                 (8 vCPU, 16GB RAM, 400GB SSD)                     │
│                        $6.99/mês                                  │
├──────────────────────────────────────────────────────────────────┤
│                                                                   │
│   ┌──────────────────────────────────────────────────────────┐   │
│   │            Docker Compose Stack                           │   │
│   ├──────────────────────────────────────────────────────────┤   │
│   │                                                           │   │
│   │  [NGINX]  →  [API FastAPI]  →  [Orquestrador]      │   │
│   │                                                           │   │
│   │  [Agente Batedor]    [Agente Analista]             │   │
│   │  [Motor Persuasão]   [Agente Escriba]              │   │
│   │  [Agente Execução]   [Agente Monitoramento]        │   │
│   │  [Agente Aprendizagem]                             │   │
│   │                                                           │   │
│   │  [PostgreSQL]  [MongoDB]  [Redis]  [ChromaDB]      │   │
│   │                                                           │   │
│   │  [Grafana]  [Prometheus]  [Portainer]              │   │
│   │                                                           │   │
│   └──────────────────────────────────────────────────────────┘   │
│                                                                   │
└──────────────────────────────────────────────────────────────────┘
                              │
                              ▼
          ┌──────────────────────────────────────────┐
          │      Serviços Externos Mínimos            │
          ├──────────────────────────────────────────┤
          │  • Proxies Residenciais ($30/mês)    │
          │  • OpenAI API (uso seletivo, $20/mês) │
          │  • CloudFlare Free (CDN + DNS)            │
          │  • Telegram Bot API (notificações)        │
          └──────────────────────────────────────────┘
```

1.2. Especificações do Servidor

Provedor Recomendado: Contabo VPS M

- **Especificações:**
  - 8 vCPU cores
  - 16 GB RAM
  - 400 GB SSD NVMe
  - Tráfego ilimitado (32 TB)
  - 1 Gbps de banda
- **Custo:** $6.99/mês (€6.50/mês)
- **Localização:** Escolher datacenter mais próximo do Brasil (EUA ou Europa)

Alternativas Comparáveis:

Provedor	Plano	vCPU	RAM	Armazenamento	Custo/mês
Contabo	VPS M	8	16GB	400GB SSD	$6.99
Hetzner	CX41	4	16GB	160GB SSD	€15.30 (~$16)
OVH	VPS Comfort	4	8GB	160GB SSD	$20
DigitalOcean	Basic 16GB	4	16GB	320GB SSD	$96

> **Recomendação:** Contabo oferece o melhor custo-benefício para esta aplicação.

---

2. Stack Tecnológica Econômica

2.1. Substituições de Serviços Gerenciados

Componente Original	Custo Original	Substituição Econômica	Cust
AWS RDS PostgreSQL	$50/mês	PostgreSQL em Docker	$0
MongoDB Atlas	$57/mês	MongoDB em Docker	$0
AWS ElastiCache (Redis)	$30/mês	Redis em Docker	$0
Pinecone	$70/mês	ChromaDB (local)	$0
AWS S3	$20/mês	Armazenamento local + Backblaze B2 (backup)	$1/r
ELK Stack gerenciado	$100/mês	Logs locais + Loki	$0
SendGrid	$15/mês	SMTP gratuito (Gmail/Brevo)	$0
Twilio SMS	$15/mês	Telegram Bot API	$0
CloudFlare Pro	$20/mês	CloudFlare Free	$0
TOTAL	**$377/mês**	**$1/mês**	**$37**

2.2. Otimização de APIs de IA

Estratégia de Uso Híbrido de LLMs

Em vez de usar exclusivamente GPT-4 ($0.03/1K tokens de entrada, $0.06/1K tokens de saída), implementamos uma estratégia híbrida:

Modelo de 3 Camadas:

1. **Camada Local (70% das requisições) - CUSTO: $0**
   - **Llama 3.1 8B** rodando localmente via Ollama
   - Uso: Análise de texto simples, classificação, extração de palavras-chave
   - Latência: ~2-5 segundos
   - Qualidade: Suficiente para tarefas básicas
2. **Camada API Econômica (25% das requisições) - CUSTO: $15/mês**
   - **GPT-4o-mini** ($0.00015/1K tokens entrada, $0.0006/1K saída)
   - Uso: Geração de mensagens menos críticas, análise intermediária
   - Economia: 200x mais barato que GPT-4
3. **Camada Premium (5% das requisições) - CUSTO: $5/mês**

- **GPT-4** (uso seletivo)
- Uso: Apenas para mensagens de abertura dos perfis TOP 10% (mais promissores)
- Justificativa: Maximizar taxa de conversão onde importa

Exemplo de Economia:

- **Cenário:** 50 perfis contatados/dia, 30 dias/mês = 1.500 mensagens/mês
- **Custo GPT-4 puro:** ~$200-300/mês
- **Custo híbrido:** ~$20/mês
- **Economia:** $180-280/mês (90%)

Modelos de Visão Computacional

- **Original:** OpenAI Vision API ($0.01275/imagem)
- **Substituição:** CLIP rodando localmente (Hugging Face)
- **Custo:** $0
- **Economia:** ~$20-40/mês (dependendo do volume)

Modelos de NLP

- **Original:** OpenAI Embeddings ($0.00002/1K tokens)
- **Substituição:** Sentence-Transformers (all-MiniLM-L6-v2) local
- **Custo:** $0
- **Economia:** ~$5-10/mês

2.3. Proxies e Anti-Detecção

Estratégia de Proxies Econômica

Opção 1: Proxies Residenciais Baratos

Provedor	Tipo	Preço	Banda	Custo/GB
Webshare	Residencial	$2.99/GB	Pay-as-you-go	$2.99
IPRoyal	Residencial	$1.75/GB	Pay-as-you-go	$1.75
Proxy-Cheap	Residencial	$3/GB	Pay-as-you-go	$3.00
BrightData	Residencial	$10/GB	Pay-as-you-go	$10.00

Recomendação: IPRoyal ($1.75/GB)

- **Consumo estimado:** 10-15 GB/mês (operação lenta e furtiva)
- **Custo:** $17.50 - 26.25/mês
- **Economia vs. BrightData:** ~$470/mês (95%)

Opção 2: Proxies Datacenter + Rotação Inteligente

Para reduzir ainda mais custos, use proxies datacenter ($0.10-0.30/proxy/mês) para 80% das operações e proxies residenciais apenas para ações críticas:

- **Proxies Datacenter (Webshare):** 10 proxies × $0.25 = $2.50/mês
- **Proxies Residenciais (IPRoyal):** 3 GB × $1.75 = $5.25/mês
- **Total:** $7.75/mês
- **Economia vs. BrightData:** ~$492/mês (98%)

Estratégia de Uso:

- **Datacenter (80%):** Navegação, coleta de URLs, visualização de perfis
- **Residencial (20%):** Login, envio de mensagens, ações sensíveis

---

3. Guia de Implantação Passo a Passo

3.1. Preparação do Servidor

Passo 1: Aquisição e Configuração Inicial

Bash

```bash
# 1. Adquirir VPS na Contabo (https://contabo.com)
# Escolher: VPS M, Ubuntu 22.04 LTS, Datacenter EUA/Europa

# 2. Conectar via SSH
ssh root@SEU_IP_DO_SERVIDOR

# 3. Atualizar sistema
apt update && apt upgrade -y

# 4. Configurar firewall (UFW)
ufw allow 22/tcp    # SSH
ufw allow 80/tcp    # HTTP
ufw allow 443/tcp   # HTTPS
ufw enable
```

```bash
# 5. Criar usuário não-root
adduser quimera
usermod -aG sudo quimera
usermod -aG docker quimera

# 6. Configurar chaves SSH (opcional, mas recomendado)
mkdir -p /home/quimera/.ssh
cp ~/.ssh/authorized_keys /home/quimera/.ssh/
chown -R quimera:quimera /home/quimera/.ssh
chmod 700 /home/quimera/.ssh
chmod 600 /home/quimera/.ssh/authorized_keys
```

Passo 2: Instalação do Docker e Docker Compose

Bash

```bash
# Trocar para usuário quimera
su - quimera

# Instalar Docker
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# Instalar Docker Compose
sudo curl -L
"https://github.com/docker/compose/releases/latest/download/docker-
compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
sudo chmod +x /usr/local/bin/docker-compose

# Verificar instalação
docker --version
docker-compose --version

# Habilitar Docker no boot
sudo systemctl enable docker
sudo systemctl start docker
```

Passo 3: Otimização do Sistema

Bash

```bash
# Aumentar limites de arquivos abertos (necessário para muitas conexões)
sudo tee -a /etc/security/limits.conf << EOF
* soft nofile 65536
* hard nofile 65536
EOF
```

```bash
# Otimizar parâmetros do kernel
sudo tee -a /etc/sysctl.conf << EOF
# Otimizações de rede
net.core.somaxconn = 65535
net.ipv4.tcp_max_syn_backlog = 8192
net.ipv4.ip_local_port_range = 1024 65535
net.ipv4.tcp_tw_reuse = 1

# Otimizações de memória
vm.swappiness = 10
vm.overcommit_memory = 1
EOF

sudo sysctl -p
```

3.2. Estrutura de Diretórios

Bash

```bash
# Criar estrutura de diretórios
mkdir -p ~/projeto_quimera/{
    docker,
    data/{postgres,mongodb,redis,chromadb,logs,backups,uploads},
    config,
    scripts,
    app/{agentes,servicos,modelos,utils}
}

cd ~/projeto_quimera
```

3.3. Docker Compose - Stack Completa

Criar arquivo `docker-compose.yml` :

YAML

```yaml
version: '3.8'

services:
  # ===== BANCO DE DADOS =====

  postgres:
    image: postgres:15-alpine
    container_name: quimera_postgres
    restart: unless-stopped
```

```yaml
    environment:
      POSTGRES_DB: quimera
      POSTGRES_USER: quimera_user
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    volumes:
      - ./data/postgres:/var/lib/postgresql/data
      - ./scripts/init_db.sql:/docker-entrypoint-initdb.d/init.sql
    ports:
      - "5432:5432"
    networks:
      - quimera_network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U quimera_user"]
      interval: 10s
      timeout: 5s
      retries: 5
    deploy:
      resources:
        limits:
          memory: 2G
        reservations:
          memory: 512M

  mongodb:
    image: mongo:7
    container_name: quimera_mongodb
    restart: unless-stopped
    environment:
      MONGO_INITDB_ROOT_USERNAME: quimera_user
      MONGO_INITDB_ROOT_PASSWORD: ${MONGO_PASSWORD}
      MONGO_INITDB_DATABASE: quimera
    volumes:
      - ./data/mongodb:/data/db
    ports:
      - "27017:27017"
    networks:
      - quimera_network
    healthcheck:
      test: echo 'db.runCommand("ping").ok' | mongosh localhost:27017/test --
quiet
      interval: 10s
      timeout: 5s
      retries: 5
    deploy:
      resources:
        limits:
          memory: 1G
        reservations:
```

```yaml
        memory: 256M

  redis:
    image: redis:7-alpine
    container_name: quimera_redis
    restart: unless-stopped
    command: redis-server --appendonly yes --requirepass ${REDIS_PASSWORD}
    volumes:
      - ./data/redis:/data
    ports:
      - "6379:6379"
    networks:
      - quimera_network
    healthcheck:
      test: ["CMD", "redis-cli", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5
    deploy:
      resources:
        limits:
          memory: 512M
        reservations:
          memory: 128M

  chromadb:
    image: chromadb/chroma:latest
    container_name: quimera_chromadb
    restart: unless-stopped
    environment:
      IS_PERSISTENT: "TRUE"
      ANONYMIZED_TELEMETRY: "FALSE"
    volumes:
      - ./data/chromadb:/chroma/chroma
    ports:
      - "8000:8000"
    networks:
      - quimera_network
    deploy:
      resources:
        limits:
          memory: 1G
        reservations:
          memory: 256M

  # ===== APLICAÇÃO =====

  api:
```

```
    build:
      context: ./app
      dockerfile: Dockerfile.api
    container_name: quimera_api
    restart: unless-stopped
    environment:
      -
DATABASE_URL=postgresql://quimera_user:${POSTGRES_PASSWORD}@postgres:5432/qui
mera
      -
MONGODB_URL=mongodb://quimera_user:${MONGO_PASSWORD}@mongodb:27017/quimera
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      - CHROMADB_URL=http://chromadb:8000
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - OLLAMA_URL=http://ollama:11434
    volumes:
      - ./app:/app
      - ./data/uploads:/app/uploads
      - ./data/logs:/app/logs
    ports:
      - "8001:8000"
    depends_on:
      - postgres
      - mongodb
      - redis
      - chromadb
    networks:
      - quimera_network
    deploy:
      resources:
        limits:
          memory: 2G
        reservations:
          memory: 512M

  orquestrador:
    build:
      context: ./app
      dockerfile: Dockerfile.orquestrador
    container_name: quimera_orquestrador
    restart: unless-stopped
    environment:
      -
DATABASE_URL=postgresql://quimera_user:${POSTGRES_PASSWORD}@postgres:5432/qui
mera
      -
MONGODB_URL=mongodb://quimera_user:${MONGO_PASSWORD}@mongodb:27017/quimera
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
```

```yaml
    volumes:
      - ./app:/app
      - ./data/logs:/app/logs
    depends_on:
      - postgres
      - mongodb
      - redis
      - api
    networks:
      - quimera_network
    deploy:
      resources:
        limits:
          memory: 1G
        reservations:
          memory: 256M

  agente_batedor:
    build:
      context: ./app
      dockerfile: Dockerfile.agente
    container_name: quimera_agente_batedor
    restart: unless-stopped
    environment:
      - AGENTE_TIPO=batedor
      - 
DATABASE_URL=postgresql://quimera_user:${POSTGRES_PASSWORD}@postgres:5432/qui
mera
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      - PROXY_URL=${PROXY_URL}
    volumes:
      - ./app:/app
      - ./data/logs:/app/logs
      - /dev/shm:/dev/shm  # Necessário para Selenium
    depends_on:
      - postgres
      - redis
    networks:
      - quimera_network
    deploy:
      resources:
        limits:
          memory: 2G
        reservations:
          memory: 512M

  agente_analista:
    build:
```

```
      context: ./app
      dockerfile: Dockerfile.agente
    container_name: quimera_agente_analista
    restart: unless-stopped
    environment:
      - AGENTE_TIPO=analista
      -
DATABASE_URL=postgresql://quimera_user:${POSTGRES_PASSWORD}@postgres:5432/qui
mera
      -
MONGODB_URL=mongodb://quimera_user:${MONGO_PASSWORD}@mongodb:27017/quimera
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      - CHROMADB_URL=http://chromadb:8000
      - OLLAMA_URL=http://ollama:11434
    volumes:
      - ./app:/app
      - ./data/logs:/app/logs
    depends_on:
      - postgres
      - mongodb
      - redis
      - chromadb
      - ollama
    networks:
      - quimera_network
    deploy:
      resources:
        limits:
          memory: 3G
        reservations:
          memory: 1G

  agente_escriba:
    build:
      context: ./app
      dockerfile: Dockerfile.agente
    container_name: quimera_agente_escriba
    restart: unless-stopped
    environment:
      - AGENTE_TIPO=escriba
      -
DATABASE_URL=postgresql://quimera_user:${POSTGRES_PASSWORD}@postgres:5432/qui
mera
      -
MONGODB_URL=mongodb://quimera_user:${MONGO_PASSWORD}@mongodb:27017/quimera
      - OPENAI_API_KEY=${OPENAI_API_KEY}
      - OLLAMA_URL=http://ollama:11434
    volumes:
```

```
      - ./app:/app
      - ./data/logs:/app/logs
    depends_on:
      - postgres
      - mongodb
      - ollama
    networks:
      - quimera_network
    deploy:
      resources:
        limits:
          memory: 2G
        reservations:
          memory: 512M

  agente_execucao:
    build:
      context: ./app
      dockerfile: Dockerfile.agente
    container_name: quimera_agente_execucao
    restart: unless-stopped
    environment:
      - AGENTE_TIPO=execucao
      -
DATABASE_URL=postgresql://quimera_user:${POSTGRES_PASSWORD}@postgres:5432/qui
mera
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      - PROXY_URL=${PROXY_URL}
    volumes:
      - ./app:/app
      - ./data/logs:/app/logs
      - /dev/shm:/dev/shm
    depends_on:
      - postgres
      - redis
    networks:
      - quimera_network
    deploy:
      resources:
        limits:
          memory: 2G
        reservations:
          memory: 512M

  agente_monitoramento:
    build:
      context: ./app
      dockerfile: Dockerfile.agente
```

```
    container_name: quimera_agente_monitoramento
    restart: unless-stopped
    environment:
      - AGENTE_TIPO=monitoramento
      -
DATABASE_URL=postgresql://quimera_user:${POSTGRES_PASSWORD}@postgres:5432/qui
mera
      -
MONGODB_URL=mongodb://quimera_user:${MONGO_PASSWORD}@mongodb:27017/quimera
      - REDIS_URL=redis://:${REDIS_PASSWORD}@redis:6379/0
      - PROXY_URL=${PROXY_URL}
      - OLLAMA_URL=http://ollama:11434
    volumes:
      - ./app:/app
      - ./data/logs:/app/logs
      - /dev/shm:/dev/shm
    depends_on:
      - postgres
      - mongodb
      - redis
      - ollama
    networks:
      - quimera_network
    deploy:
      resources:
        limits:
          memory: 2G
        reservations:
          memory: 512M

  agente_aprendizagem:
    build:
      context: ./app
      dockerfile: Dockerfile.agente
    container_name: quimera_agente_aprendizagem
    restart: unless-stopped
    environment:
      - AGENTE_TIPO=aprendizagem
      -
DATABASE_URL=postgresql://quimera_user:${POSTGRES_PASSWORD}@postgres:5432/qui
mera
      - CHROMADB_URL=http://chromadb:8000
    volumes:
      - ./app:/app
      - ./data/logs:/app/logs
      - ./data/modelos:/app/modelos
    depends_on:
      - postgres
```

```yaml
      - chromadb
    networks:
      - quimera_network
    deploy:
      resources:
        limits:
          memory: 2G
        reservations:
          memory: 512M

  # ===== MODELOS LOCAIS =====

  ollama:
    image: ollama/ollama:latest
    container_name: quimera_ollama
    restart: unless-stopped
    volumes:
      - ./data/ollama:/root/.ollama
    ports:
      - "11434:11434"
    networks:
      - quimera_network
    deploy:
      resources:
        limits:
          memory: 4G
        reservations:
          memory: 2G

  # ===== MONITORAMENTO =====

  prometheus:
    image: prom/prometheus:latest
    container_name: quimera_prometheus
    restart: unless-stopped
    volumes:
      - ./config/prometheus.yml:/etc/prometheus/prometheus.yml
      - ./data/prometheus:/prometheus
    ports:
      - "9090:9090"
    networks:
      - quimera_network
    deploy:
      resources:
        limits:
          memory: 512M
        reservations:
          memory: 128M
```

```yaml
  grafana:
    image: grafana/grafana:latest
    container_name: quimera_grafana
    restart: unless-stopped
    environment:
      - GF_SECURITY_ADMIN_PASSWORD=${GRAFANA_PASSWORD}
      - GF_INSTALL_PLUGINS=redis-datasource
    volumes:
      - ./data/grafana:/var/lib/grafana
      - ./config/grafana/dashboards:/etc/grafana/provisioning/dashboards
      - ./config/grafana/datasources:/etc/grafana/provisioning/datasources
    ports:
      - "3000:3000"
    depends_on:
      - prometheus
    networks:
      - quimera_network
    deploy:
      resources:
        limits:
          memory: 512M
        reservations:
          memory: 128M

  # ===== PROXY REVERSO =====

  nginx:
    image: nginx:alpine
    container_name: quimera_nginx
    restart: unless-stopped
    volumes:
      - ./config/nginx.conf:/etc/nginx/nginx.conf
      - ./config/ssl:/etc/nginx/ssl
      - ./data/logs/nginx:/var/log/nginx
    ports:
      - "80:80"
      - "443:443"
    depends_on:
      - api
      - grafana
    networks:
      - quimera_network
    deploy:
      resources:
        limits:
          memory: 256M
        reservations:
```

```yaml
          memory: 64M

  # ===== GERENCIAMENTO =====

  portainer:
    image: portainer/portainer-ce:latest
    container_name: quimera_portainer
    restart: unless-stopped
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
      - ./data/portainer:/data
    ports:
      - "9443:9443"
    networks:
      - quimera_network
    deploy:
      resources:
        limits:
          memory: 256M
        reservations:
          memory: 64M

networks:
  quimera_network:
    driver: bridge

volumes:
  postgres_data:
  mongodb_data:
  redis_data:
  chromadb_data:
  ollama_data:
  prometheus_data:
  grafana_data:
  portainer_data:
```

3.4. Arquivo de Variáveis de Ambiente

Criar arquivo `.env`:

```bash
# Banco de Dados
POSTGRES_PASSWORD=senha_super_segura_postgres_123
MONGO_PASSWORD=senha_super_segura_mongo_456
REDIS_PASSWORD=senha_super_segura_redis_789
```

```
# APIs Externas
OPENAI_API_KEY=sk-proj-xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
PROXY_URL=http://usuario:senha@proxy.iproyal.com:12321

# Grafana
GRAFANA_PASSWORD=admin_grafana_senha_123

# Telegram Bot (Notificações)
TELEGRAM_BOT_TOKEN=123456789:ABCdefGHIjklMNOpqrsTUVwxyz
TELEGRAM_CHAT_ID=123456789

# Configurações do Sistema
AMBIENTE=producao
DEBUG=false
LOG_LEVEL=info
```

3.5. Inicialização do Sistema

Bash

```bash
# 1. Baixar modelos do Ollama
docker-compose up -d ollama
docker exec -it quimera_ollama ollama pull llama3.1:8b
docker exec -it quimera_ollama ollama pull nomic-embed-text

# 2. Inicializar bancos de dados
docker-compose up -d postgres mongodb redis chromadb

# Aguardar 30 segundos para os bancos inicializarem
sleep 30

# 3. Executar migrações do banco de dados
docker-compose run --rm api python manage.py migrate

# 4. Criar usuário admin
docker-compose run --rm api python manage.py create_superuser

# 5. Iniciar todos os serviços
docker-compose up -d

# 6. Verificar status
docker-compose ps

# 7. Visualizar logs
docker-compose logs -f --tail=100
```

4. Otimizações Avançadas de Custo

4.1. Estratégia de Uso de LLM em 3 Camadas

Implementação do Roteador de LLM

Criar arquivo `app/servicos/llm_router.py` :

Python

```python
from enum import Enum
from typing import Optional
import ollama
from openai import OpenAI

class LLMTier(Enum):
    LOCAL = "local"        # Llama 3.1 8B (gratuito)
    ECONOMICO = "economico"  # GPT-4o-mini ($0.00015/1K tokens)
    PREMIUM = "premium"    # GPT-4 ($0.03/1K tokens)

class LLMRouter:
    def __init__(self):
        self.ollama_client = ollama.Client(host='http://ollama:11434')
        self.openai_client = OpenAI()

    def selecionar_tier(self, tarefa: str, pontuacao_perfil: float) ->
LLMTier:
        """
        Seleciona o tier de LLM baseado na tarefa e importância.

        Regras:
        - LOCAL: Análise básica, classificação, extração de keywords
        - ECONOMICO: Geração de mensagens para perfis médios (pontuação 6-8)
        - PREMIUM: Geração de mensagens para perfis top (pontuação > 8)
        """
        if tarefa in ["analise_texto", "classificacao", "extracao_keywords"]:
            return LLMTier.LOCAL

        elif tarefa == "geracao_mensagem":
            if pontuacao_perfil >= 8.5:
                return LLMTier.PREMIUM
            elif pontuacao_perfil >= 7.0:
                return LLMTier.ECONOMICO
            else:
                return LLMTier.LOCAL

        else:
```

```python
            return LLMTier.ECONOMICO

    def gerar_texto(
        self,
        prompt: str,
        tier: Optional[LLMTier] = None,
        max_tokens: int = 200
    ) -> str:
        """Gera texto usando o tier apropriado."""

        if tier == LLMTier.LOCAL:
            response = self.ollama_client.generate(
                model='llama3.1:8b',
                prompt=prompt,
                options={'num_predict': max_tokens}
            )
            return response['response']

        elif tier == LLMTier.ECONOMICO:
            response = self.openai_client.chat.completions.create(
                model="gpt-4o-mini",
                messages=[{"role": "user", "content": prompt}],
                max_tokens=max_tokens,
                temperature=0.7
            )
            return response.choices[0].message.content

        elif tier == LLMTier.PREMIUM:
            response = self.openai_client.chat.completions.create(
                model="gpt-4",
                messages=[{"role": "user", "content": prompt}],
                max_tokens=max_tokens,
                temperature=0.7
            )
            return response.choices[0].message.content

        else:
            raise ValueError(f"Tier inválido: {tier}")

# Exemplo de uso
router = LLMRouter()

# Análise de texto (usa Llama local - GRÁTIS)
tier = router.selecionar_tier("analise_texto", pontuacao_perfil=7.5)
resultado = router.gerar_texto(
    prompt="Extraia os interesses desta biografia: ...",
    tier=tier
)
```

```python
# Geração de mensagem para perfil top (usa GPT-4 - PREMIUM)
tier = router.selecionar_tier("geracao_mensagem", pontuacao_perfil=9.2)
mensagem = router.gerar_texto(
    prompt="Gere uma mensagem de abertura para este perfil: ...",
    tier=tier
)
```

Economia Estimada

Tarefa	Volume/mês	Tier Original	Custo Original	Tier Novo	Custo Novo
Análise de texto	1.500 perfis	GPT-4	$45	LOCAL	$0
Extração keywords	1.500 perfis	GPT-4	$30	LOCAL	$0
Análise sentimento	500 respostas	GPT-4	$15	LOCAL	$0
Mensagens médias	1.200 msgs	GPT-4	$180	ECONOMICO	$2
Mensagens top	300 msgs	GPT-4	$45	PREMIUM	$45
TOTAL	-	-	**$315**	-	**$47**

4.2. Cache Inteligente

Implementar cache em múltiplas camadas para reduzir chamadas a APIs:

Python

```python
# app/servicos/cache_manager.py
import redis
import hashlib
import json
from typing import Optional, Any
from functools import wraps

class CacheManager:
    def __init__(self, redis_url: str):
        self.redis = redis.from_url(redis_url)

    def cache_key(self, prefix: str, *args, **kwargs) -> str:
        """Gera chave de cache determinística."""
        data = json.dumps({"args": args, "kwargs": kwargs}, sort_keys=True)
        hash_value = hashlib.md5(data.encode()).hexdigest()
```

```python
        return f"{prefix}:{hash_value}"

    def get(self, key: str) -> Optional[Any]:
        """Recupera valor do cache."""
        value = self.redis.get(key)
        if value:
            return json.loads(value)
        return None

    def set(self, key: str, value: Any, ttl: int = 3600):
        """Armazena valor no cache com TTL."""
        self.redis.setex(key, ttl, json.dumps(value))

    def cached(self, prefix: str, ttl: int = 3600):
        """Decorator para cachear resultados de funções."""
        def decorator(func):
            @wraps(func)
            def wrapper(*args, **kwargs):
                key = self.cache_key(prefix, *args, **kwargs)

                # Tentar recuperar do cache
                cached_value = self.get(key)
                if cached_value is not None:
                    return cached_value

                # Executar função e cachear resultado
                result = func(*args, **kwargs)
                self.set(key, result, ttl)
                return result

            return wrapper
        return decorator

# Exemplo de uso
cache = CacheManager(redis_url="redis://:senha@redis:6379/0")

@cache.cached(prefix="analise_perfil", ttl=86400)  # 24 horas
def analisar_perfil(url_perfil: str):
    """
    Análise de perfil é cara (NLP + Vision).
    Cachear por 24h evita reprocessamento.
    """
    # ... lógica de análise ...
    return resultado

@cache.cached(prefix="embeddings", ttl=604800)  # 7 dias
def gerar_embeddings(texto: str):
    """
```

```
    Embeddings raramente mudam.
    Cachear por 7 dias.
    """
    # ... gerar embeddings ...
    return embeddings
```

Economia Estimada:

- **Análises de perfil repetidas:** ~30% dos perfis são revisitados
- **Economia:** $15-30/mês em chamadas de API evitadas

4.3. Processamento em Lote (Batch Processing)

Agrupar operações para reduzir overhead:

Python

```python
# app/servicos/batch_processor.py
from typing import List, Callable, Any
import asyncio

class BatchProcessor:
    def __init__(self, batch_size: int = 50, delay: int = 5):
        self.batch_size = batch_size
        self.delay = delay
        self.queue = []

    async def add(self, item: Any):
        """Adiciona item à fila de processamento."""
        self.queue.append(item)

        if len(self.queue) >= self.batch_size:
            await self.process_batch()

    async def process_batch(self):
        """Processa lote de itens."""
        if not self.queue:
            return

        batch = self.queue[:self.batch_size]
        self.queue = self.queue[self.batch_size:]

        # Processar em paralelo
        tasks = [self.process_item(item) for item in batch]
        await asyncio.gather(*tasks)

    async def process_item(self, item: Any):
        """Processa item individual."""
```

```python
        # Implementar lógica específica
        pass

# Exemplo: Análise de perfis em lote
batch_processor = BatchProcessor(batch_size=50)

for url in urls_perfis:
    await batch_processor.add(url)

# Processar itens restantes
await batch_processor.process_batch()
```

Economia Estimada:

- **Redução de overhead:** ~20% mais eficiente
- **Economia de recursos:** Permite usar servidor menor

4.4. Agendamento Inteligente

Executar tarefas pesadas em horários de baixo custo:

Python

```python
# app/servicos/scheduler.py
from datetime import datetime, time
from apscheduler.schedulers.asyncio import AsyncIOScheduler

class SmartScheduler:
    def __init__(self):
        self.scheduler = AsyncIOScheduler()

    def agendar_tarefa_pesada(self, func, horario_preferencial: time):
        """
        Agenda tarefas pesadas para horários de menor carga.

        Estratégia:
        - Análise de perfis: 02:00-06:00 (madrugada)
        - Treinamento de modelos: 03:00-05:00
        - Backups: 04:00
        """
        self.scheduler.add_job(
            func,
            trigger='cron',
            hour=horario_preferencial.hour,
            minute=horario_preferencial.minute
        )

    def start(self):
```

```
        self.scheduler.start()

# Exemplo de uso
scheduler = SmartScheduler()

# Análise de perfis às 3h da manhã
scheduler.agendar_tarefa_pesada(
    func=analisar_perfis_pendentes,
    horario_preferencial=time(hour=3, minute=0)
)

# Treinamento de modelos às 4h da manhã
scheduler.agendar_tarefa_pesada(
    func=treinar_modelo_recompensa,
    horario_preferencial=time(hour=4, minute=0)
)

scheduler.start()
```

---

5. Monitoramento e Observabilidade Econômica

5.1. Stack de Monitoramento Gratuita

Grafana Cloud (Gratuito)

- **Plano Free:** 10k séries métricas, 50GB logs, 50GB traces
- **Custo:** $0/mês
- **Uso:** Dashboards, alertas, visualizações

Prometheus (Local)

- **Coleta de métricas:** CPU, memória, disco, rede
- **Métricas customizadas:** Taxa de resposta, tempo de processamento, etc.
- **Custo:** $0 (roda no mesmo servidor)

Configuração do Prometheus

Criar arquivo `config/prometheus.yml` :

YAML

```yaml
global:
  scrape_interval: 15s
```

```yaml
  evaluation_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['localhost:9090']

  - job_name: 'api'
    static_configs:
      - targets: ['api:8000']

  - job_name: 'postgres'
    static_configs:
      - targets: ['postgres-exporter:9187']

  - job_name: 'redis'
    static_configs:
      - targets: ['redis-exporter:9121']

  - job_name: 'node'
    static_configs:
      - targets: ['node-exporter:9100']
```

Dashboard Grafana Customizado

Criar dashboard com métricas-chave:

- **Taxa de Interação Automatizada (TIA):** % de perfis contatados
- **Taxa de Resposta Positiva (TRP):** % de respostas positivas
- **Taxa de Sucesso Final (TSF):** % de conversões (WhatsApp obtido)
- **Custo por Conversão:** Custo total / conversões
- **Uso de Recursos:** CPU, RAM, disco, rede
- **Custos de API:** Gastos com OpenAI, proxies, etc.

5.2. Alertas Inteligentes

Configurar alertas para evitar custos inesperados:

YAML

```yaml
# config/grafana/alerts.yml
groups:
  - name: custos
    interval: 5m
    rules:
```

```
      - alert: CustoOpenAIAlto
        expr: sum(openai_api_cost_total) > 50
        for: 1h
        labels:
          severity: warning
        annotations:
          summary: "Custo da API OpenAI ultrapassou $50 no último hora"
          description: "Revisar uso de GPT-4. Considerar migrar para GPT-4o-
mini."

      - alert: ConsumoProxyAlto
        expr: sum(proxy_bandwidth_gb) > 15
        for: 1d
        labels:
          severity: warning
        annotations:
          summary: "Consumo de proxy ultrapassou 15GB no último dia"
          description: "Revisar estratégia de scraping. Considerar reduzir
frequência."

      - alert: TaxaErroAlta
        expr: rate(http_requests_total{status=~"5.."}[5m]) > 0.05
        for: 10m
        labels:
          severity: critical
        annotations:
          summary: "Taxa de erro HTTP 5xx acima de 5%"
          description: "Verificar logs e saúde dos serviços."
```

---

6. Estratégias de Backup Econômicas

6.1. Backup Local + Cloud Econômico

Estratégia de 3-2-1

- **3 cópias:** Original + 2 backups
- **2 mídias diferentes:** Disco local + cloud
- **1 cópia offsite:** Backblaze B2

Implementação

Bash

```bash
# Script de backup automático
# Salvar em: ~/projeto_quimera/scripts/backup.sh

#!/bin/bash

BACKUP_DIR="/home/quimera/projeto_quimera/data/backups"
DATE=$(date +%Y%m%d_%H%M%S)
BACKUP_NAME="quimera_backup_${DATE}"

# 1. Backup PostgreSQL
docker exec quimera_postgres pg_dumpall -U quimera_user | gzip >
"${BACKUP_DIR}/${BACKUP_NAME}_postgres.sql.gz"

# 2. Backup MongoDB
docker exec quimera_mongodb mongodump --archive | gzip >
"${BACKUP_DIR}/${BACKUP_NAME}_mongodb.archive.gz"

# 3. Backup Redis (RDB snapshot)
docker exec quimera_redis redis-cli --no-auth-warning -a ${REDIS_PASSWORD}
BGSAVE
sleep 5
docker cp quimera_redis:/data/dump.rdb
"${BACKUP_DIR}/${BACKUP_NAME}_redis.rdb"

# 4. Backup ChromaDB
tar -czf "${BACKUP_DIR}/${BACKUP_NAME}_chromadb.tar.gz" -C
/home/quimera/projeto_quimera/data chromadb/

# 5. Backup modelos ML
tar -czf "${BACKUP_DIR}/${BACKUP_NAME}_modelos.tar.gz" -C
/home/quimera/projeto_quimera/data modelos/

# 6. Limpar backups antigos (manter últimos 7 dias localmente)
find "${BACKUP_DIR}" -name "quimera_backup_*" -mtime +7 -delete

# 7. Enviar para Backblaze B2 (apenas backups diários)
if [ $(date +%H) -eq 4 ]; then
    b2 sync --delete --replaceNewer "${BACKUP_DIR}" b2://meu-bucket-
quimera/backups/
fi

echo "Backup concluído: ${BACKUP_NAME}"
```

Configurar Cron

Bash

```bash
# Adicionar ao crontab
crontab -e

# Backup a cada 6 horas
0 */6 * * * /home/quimera/projeto_quimera/scripts/backup.sh >>
/home/quimera/projeto_quimera/data/logs/backup.log 2>&1
```

Backblaze B2 (Cloud Storage Econômico)

- **Custo:** $0.005/GB/mês (armazenamento) + $0.01/GB (download)
- **Exemplo:** 50GB de backups = $0.25/mês
- **Comparação:** AWS S3 = $1.15/mês (4.6x mais caro)

Instalação do B2 CLI:

```bash
pip3 install b2
b2 authorize-account <applicationKeyId> <applicationKey>
b2 create-bucket meu-bucket-quimera allPrivate
```

---

7. Otimização de Performance

7.1. Tuning do PostgreSQL

Editar `config/postgresql.conf`:

```plain text
# Otimizações para servidor com 16GB RAM

# Memória
shared_buffers = 4GB
effective_cache_size = 12GB
maintenance_work_mem = 1GB
work_mem = 64MB

# Checkpoints
checkpoint_completion_target = 0.9
wal_buffers = 16MB
default_statistics_target = 100

# Planner
```

```
random_page_cost = 1.1  # Para SSD
effective_io_concurrency = 200

# Logging (reduzir para economizar I/O)
log_min_duration_statement = 1000  # Log apenas queries > 1s
log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h '
log_checkpoints = on
log_connections = off
log_disconnections = off
log_lock_waits = on
```

7.2. Tuning do MongoDB

Editar `config/mongod.conf`:

YAML

```yaml
# Otimizações para servidor com 16GB RAM

storage:
  dbPath: /data/db
  journal:
    enabled: true
  wiredTiger:
    engineConfig:
      cacheSizeGB: 2  # Limitar cache do WiredTiger
      journalCompressor: snappy
    collectionConfig:
      blockCompressor: snappy

systemLog:
  destination: file
  path: /var/log/mongodb/mongod.log
  logAppend: true
  logRotate: reopen

net:
  port: 27017
  bindIp: 0.0.0.0
  maxIncomingConnections: 1000

operationProfiling:
  mode: slowOp
  slowOpThresholdMs: 1000  # Log apenas operações > 1s
```

7.3. Índices Estratégicos

Criar índices apenas onde necessário para economizar espaço e I/O:

```sql
-- PostgreSQL: Índices essenciais

-- Tabela perfis
CREATE INDEX idx_perfis_status ON perfis(status_processamento);
CREATE INDEX idx_perfis_pontuacao ON perfis(pontuacao_final DESC) WHERE
status_processamento = 'qualificado';
CREATE INDEX idx_perfis_timestamp ON perfis(timestamp_coleta);

-- Tabela conversas
CREATE INDEX idx_conversas_status ON conversas(status);
CREATE INDEX idx_conversas_perfil ON conversas(perfil_id);
CREATE INDEX idx_conversas_timestamp ON conversas(timestamp_inicio);

-- Tabela mensagens
CREATE INDEX idx_mensagens_conversa ON mensagens(conversa_id);
CREATE INDEX idx_mensagens_timestamp ON mensagens(timestamp_envio);

-- Índices parciais (economizam espaço)
CREATE INDEX idx_perfis_qualificados ON perfis(pontuacao_final)
WHERE status_processamento IN ('qualificado', 'contatado');

CREATE INDEX idx_conversas_ativas ON conversas(timestamp_inicio)
WHERE status IN ('iniciada', 'aguardando_resposta', 'respondida');
```

---

8. Plano de Escalabilidade Gradual

8.1. Fase 1: MVP Mínimo ($57/mês)

Objetivo: Validar o sistema com volume reduzido

- **Servidor:** Contabo VPS M ($7/mês)
- **Proxies:** IPRoyal 10GB ($17.50/mês)
- **LLM:** Llama local + GPT-4o-mini seletivo ($10/mês)
- **Backup:** Backblaze B2 20GB ($0.10/mês)
- **Monitoramento:** Grafana Cloud Free ($0)
- **Notificações:** Telegram ($0)

Capacidade:

- 20-30 perfis contatados/dia
- 600-900 perfis/mês
- Taxa de resposta esperada: 20-30% → 120-270 respostas/mês

8.2. Fase 2: Crescimento Moderado ($120/mês)

Objetivo: Escalar para volume médio

- **Servidor:** Contabo VPS L ($15/mês) - 10 vCPU, 30GB RAM
- **Proxies:** IPRoyal 30GB ($52.50/mês)
- **LLM:** Híbrido otimizado ($30/mês)
- **Backup:** Backblaze B2 50GB ($0.25/mês)
- **Monitoramento:** Grafana Cloud Free ($0)
- **CDN:** CloudFlare Free ($0)

Capacidade:

- 50-70 perfis contatados/dia
- 1.500-2.100 perfis/mês
- Taxa de resposta esperada: 20-30% → 300-630 respostas/mês

8.3. Fase 3: Alta Escala ($300/mês)

Objetivo: Operação em larga escala

- **Servidor:** Hetzner Dedicated AX41 ($50/mês) - 12 cores, 64GB RAM
- **Proxies:** IPRoyal 100GB ($175/mês)
- **LLM:** Híbrido + GPT-4 seletivo ($50/mês)
- **Backup:** Backblaze B2 100GB ($0.50/mês)
- **Monitoramento:** Grafana Cloud Pro ($20/mês)
- **CDN:** CloudFlare Pro ($20/mês)

Capacidade:

- 150-200 perfis contatados/dia
- 4.500-6.000 perfis/mês
- Taxa de resposta esperada: 20-30% → 900-1.800 respostas/mês

---

9. Checklist de Implantação

9.1. Pré-Implantação

- [ ] Adquirir VPS (Contabo VPS M recomendado)
- [ ] Configurar DNS (apontar domínio para IP do servidor)
- [ ] Adquirir certificado SSL (Let's Encrypt gratuito)
- [ ] Criar conta Backblaze B2 (backup)
- [ ] Criar conta Grafana Cloud (monitoramento)
- [ ] Adquirir proxies (IPRoyal recomendado)
- [ ] Criar API key OpenAI (para GPT-4o-mini)
- [ ] Criar bot Telegram (notificações)
- [ ] Preparar contas de isca (Instagram/Facebook)

9.2. Implantação

- [ ] Conectar ao servidor via SSH
- [ ] Atualizar sistema operacional
- [ ] Configurar firewall (UFW)
- [ ] Instalar Docker e Docker Compose
- [ ] Clonar repositório do projeto
- [ ] Configurar variáveis de ambiente (.env)
- [ ] Baixar modelos Ollama (Llama 3.1 8B)
- [ ] Inicializar bancos de dados
- [ ] Executar migrações
- [ ] Criar usuário admin
- [ ] Iniciar todos os serviços
- [ ] Configurar NGINX com SSL
- [ ] Configurar backups automáticos
- [ ] Configurar monitoramento (Prometheus + Grafana)
- [ ] Testar conectividade com APIs externas
- [ ] Testar envio de notificações (Telegram)

9.3. Pós-Implantação

- [ ] Configurar hotspots digitais
- [ ] Definir perfil ideal (critérios)
- [ ] Ajustar pesos de pontuação
- [ ] Executar primeira varredura (Agente Batedor)
- [ ] Analisar primeiros perfis (Agente Analista)
- [ ] Gerar primeira mensagem (Agente Escriba)
- [ ] Enviar primeira mensagem (Agente de Execução)
- [ ] Monitorar primeira resposta (Agente de Monitoramento)
- [ ] Realizar primeiro handoff
- [ ] Fornecer primeiro feedback (Agente de Aprendizagem)
- [ ] Revisar métricas no dashboard
- [ ] Ajustar configurações conforme necessário

---

10. Troubleshooting e Otimização Contínua

10.1. Problemas Comuns e Soluções

Problema: Alto Uso de Memória

Sintoma: Servidor fica lento, serviços crasham

Solução:

```bash
# Verificar uso de memória
docker stats

# Reduzir limites de memória dos containers
# Editar docker-compose.yml e ajustar deploy.resources.limits.memory

# Reiniciar serviços
docker-compose restart
```

Problema: Banco de Dados Lento

Sintoma: Queries demoram muito

Solução:

Bash

```bash
# Analisar queries lentas no PostgreSQL
docker exec -it quimera_postgres psql -U quimera_user -d quimera -c "
SELECT query, calls, total_time, mean_time
FROM pg_stat_statements
ORDER BY mean_time DESC
LIMIT 10;
"

# Criar índices faltantes
# Executar VACUUM e ANALYZE
docker exec -it quimera_postgres psql -U quimera_user -d quimera -c "VACUUM
ANALYZE;"
```

Problema: Custo de API Alto

Sintoma: Gastos com OpenAI acima do esperado

Solução:

Python

```python
# Revisar uso do LLMRouter
# Aumentar % de uso do tier LOCAL
# Reduzir max_tokens nas chamadas
# Implementar cache mais agressivo

# Exemplo: Forçar uso de Llama local para mais tarefas
def selecionar_tier(self, tarefa: str, pontuacao_perfil: float) -> LLMTier:
    if tarefa in ["analise_texto", "classificacao", "extracao_keywords",
"analise_sentimento"]:
        return LLMTier.LOCAL  # Expandir uso local

    elif tarefa == "geracao_mensagem":
        if pontuacao_perfil >= 9.0:  # Aumentar limiar para GPT-4
            return LLMTier.PREMIUM
        elif pontuacao_perfil >= 7.5:  # Aumentar limiar para GPT-4o-mini
            return LLMTier.ECONOMICO
        else:
            return LLMTier.LOCAL  # Usar Llama para perfis de menor
prioridade
```

10.2. Monitoramento de Custos em Tempo Real

Criar dashboard de custos no Grafana:

Python

```python
# app/servicos/cost_tracker.py
from prometheus_client import Counter, Gauge

# Métricas de custo
openai_cost_total = Counter('openai_cost_total', 'Custo total da API OpenAI')
proxy_bandwidth_gb = Counter('proxy_bandwidth_gb', 'Banda de proxy consumida
em GB')
custo_mensal_estimado = Gauge('custo_mensal_estimado', 'Custo mensal
estimado')

def registrar_custo_openai(modelo: str, tokens_entrada: int, tokens_saida:
int):
    """Registra custo de chamada à API OpenAI."""
    if modelo == "gpt-4":
        custo = (tokens_entrada / 1000 * 0.03) + (tokens_saida / 1000 * 0.06)
    elif modelo == "gpt-4o-mini":
        custo = (tokens_entrada / 1000 * 0.00015) + (tokens_saida / 1000 *
0.0006)
    else:
        custo = 0

    openai_cost_total.inc(custo)

def registrar_uso_proxy(bytes_consumidos: int):
    """Registra uso de proxy."""
    gb = bytes_consumidos / (1024 ** 3)
    proxy_bandwidth_gb.inc(gb)

def calcular_custo_mensal_estimado():
    """Calcula custo mensal estimado baseado no uso atual."""
    custo_servidor = 7.0  # Contabo VPS M
    custo_openai = openai_cost_total._value._value
    custo_proxy = proxy_bandwidth_gb._value._value * 1.75  # IPRoyal
    custo_backup = 0.25  # Backblaze B2

    total = custo_servidor + custo_openai + custo_proxy + custo_backup
    custo_mensal_estimado.set(total)

    return total
```

11. Conclusão e Próximos Passos

11.1. Resumo de Economia

Com a arquitetura econômica proposta, conseguimos reduzir os custos operacionais de **$1.137-1.337/mês** para **$57-87/mês**, uma economia de aproximadamente **93%**, mantendo a funcionalidade essencial do sistema.

Principais Estratégias de Economia:

1. **Consolidação em VPS único** (Contabo) → Economia de $600/mês
2. **Uso de modelos locais** (Llama 3.1) → Economia de $200/mês
3. **Proxies econômicos** (IPRoyal) → Economia de $470/mês
4. **Serviços gratuitos** (Grafana Cloud, Telegram, CloudFlare) → Economia de $165/mês
5. **Backup econômico** (Backblaze B2) → Economia de $20/mês

11.2. Roadmap de Otimização Contínua

Mês 1-2: Validação e Ajuste

- Monitorar métricas de performance
- Ajustar pesos de pontuação
- Refinar estratégias de persuasão
- Otimizar uso de LLM

Mês 3-4: Otimização de Custos

- Analisar padrões de uso
- Identificar gargalos
- Implementar cache mais agressivo
- Reduzir uso de APIs pagas

Mês 5-6: Escalabilidade

- Avaliar necessidade de upgrade de servidor
- Implementar sharding de banco de dados (se necessário)
- Otimizar queries lentas
- Implementar CDN para assets estáticos

11.3. Métricas de Sucesso

KPIs Operacionais:

- Taxa de Interação Automatizada (TIA): > 95%
- Taxa de Resposta Positiva (TRP): > 20%
- Taxa de Sucesso Final (TSF): > 60% (das respostas)
- Tempo Médio de Resposta: < 6 horas

KPIs Financeiros:

- Custo por Perfil Contatado: < $0.10
- Custo por Resposta Positiva: < $0.50
- Custo por Conversão (WhatsApp): < $3.00
- ROI: > 500% (considerando valor de cada conversão)

11.4. Suporte e Recursos

Documentação:

- Guia de implantação completo (este documento)
- Documentação UML do sistema
- README com instruções de uso

Comunidade:

- Fórum de discussão (Discord/Telegram)
- Issues no GitHub
- Wiki com troubleshooting

Monitoramento:

- Dashboard Grafana com métricas em tempo real
- Alertas configurados para anomalias
- Logs centralizados para debugging

Versão: 1.0

Data: Outubro de 2025

Autor: Documentação técnica do Projeto Quimera

Licença: Uso restrito conforme acordado com o proprietário do projeto